



Shared:

FinWallet.Shared.Contracts -> ServiceResult and common transport contracts

FinWallet.Shared.Web -> Swagger, rate limiting, CORS, Kestrel limits, security headers and internal-service-key checks

Dependency direction:

Api -> Application -> Domain

Api -> Infrastructure -> Application / Domain

Gateway -> Shared.Web / Shared.Contracts

Fake APIs -> Shared.Web / Shared.Contracts

Domain knows nothing about SQL, HTTP, Redis or provider DTOs. Application does not depend on concrete Infrastructure implementations.

4. Gateway and trust-boundary topology

```
[Client]
|
| JWT
v
[Gateway]
|
| JWT + DownstreamServiceKey
v
[FinWallet.Api]

[FinWallet.Api]
|
| InternalServiceKey
v
[Gateway /providers/*]
|
| DownstreamServiceKey
v
[Fake Provider]
```

This creates four distinct control points:

- 1 Client -> Gateway: public edge authentication and rate limiting.
- 2 Gateway -> FinWallet.Api: Gateway adds a downstream service credential; the API independently validates JWT and ownership.
- 3 FinWallet.Api -> Gateway provider route: InternalServiceKey restricts provider routes to trusted internal callers.
- 4 Gateway -> Fake provider: the destination validates a separate DownstreamServiceKey.

Result: direct access to a backend port does not automatically make that request trusted.

5. YARP route and load-balancing topology

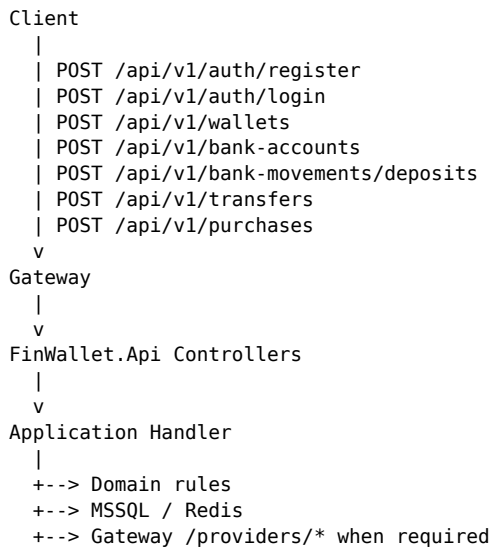
```
Client /api/*
|
v
route: finwallet-api
|
v
cluster: finwallet-api-cluster
|
+--> FinWallet.Api replica 1
+--> FinWallet.Api replica 2 (can be added in production)
+--> FinWallet.Api replica N

FinWallet.Api /providers/bank/*
|
v
route: fake-bank
|
v
cluster: fake-bank-cluster
|
+--> FakeBank replica 1
```

+--> FakeBank replica N

YARP clusters can use PowerOfTwoChoices. Destination addresses, health checks, maximum connections and timeouts are managed through appsettings. Development can use a single destination while production can add replicas to the same cluster.

6. Public API traffic topology



Anonymous routes are limited to register, registration verification, login and refresh. Other customer routes require JWT at the Gateway.

7. Provider integration topology

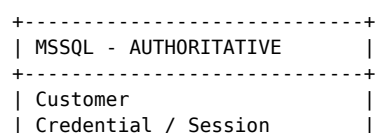


Provider responsibilities:

Service	Responsibility
FakeBank.Api	External bank account, provider transaction, statement and timeout/failure simulation
FakeFraud.Api	External Allow/Review/Deny, score and reason codes
FakeCutoff.Api	Business day, cutoff, processing and settlement dates
FakeCampaign.Api	Merchant campaign eligibility, discount and sponsor type
FakeCommunication.Api	SMS/email acceptance, delay, timeout and failure simulation

Provider DTOs do not leak into Application/Domain; Infrastructure adapters/anti-corruption layers map them.

8. Data-ownership topology



Wallet / BankAccount
FinancialTransaction
LedgerJournal / LedgerEntry
Idempotency
FraudEvent / Review
Outbox / Inbox
Reconciliation
Audit

Redis - TRANSIENT SUPPORT
OTP TTL
temporary counters
fraud velocity
hot/session cache support
temporary coordination

Redis is never the sole authority for financial correctness. Wallet balances, ledger state, completed transactions and durable idempotency remain in MSSQL.

9. Bank-to-Wallet funding topology

```

Customer Bank Account (FakeBank)
    |
    | provider withdrawal
    v
FinWallet Bank Settlement Asset
    |
    | same DB transaction
    +--> Customer Wallet Liability +1000
    +--> FinancialTransaction = Completed
    +--> LedgerJournal
    +--> LedgerEntries (Debit = Credit)
    +--> Idempotency final result
    +--> Outbox notification

```

Important distinction: BankAccount and Wallet are not the same thing. BankAccount models the external bank account; Wallet models the customer's balance inside FinWallet.

10. Wallet-to-Wallet transfer topology

```

Client
    |
    v
Gateway JWT
    |
    v
FinWallet.Api
    |
    +--> completed idempotency replay check
    +--> server-side risk signals
    +--> Internal Fraud
    +--> Gateway -> FakeFraud
    +--> Allow / Review / Deny
    |
    +--> on ALLOW, one MSSQL transaction:
        source Wallet debit
        destination Wallet credit
        FinancialTransaction
        LedgerJournal
        LedgerEntries
        durable Idempotency result

```

The external fraud HTTP call is never executed while the SQL transaction is open.

11. Purchase topology

```

Client
    |
    v

```

```

Gateway -> FinWallet.Api
|
+--> internal fraud
+--> external FakeFraud
+--> FakeCampaign
|
v
Atomic MSSQL posting
|
+--> customer wallet liability debit
+--> merchant payable credit
+--> campaign expense debit (platform-sponsored campaign)
+--> FinancialTransaction
+--> balanced Ledger
+--> Outbox

```

The campaign provider only calculates eligibility/discount information; financial accounting remains inside FinWallet.

12. Background-worker topology

```

FinWallet.Api process
|
+--> BankMoneyMovementBackgroundService
|
|   +--> due Scheduled/Pending transactions
|   +--> Gateway -> FakeBank status / processing
|   +--> durable finalization / compensation
|
+--> OutboxDispatchBackgroundService
|
|   +--> claim Outbox row
|   +--> SQL transaction closes
|   +--> Gateway -> FakeCommunication
|   +--> success/failure/backoff state

```

A notification failure never rolls back an already committed money movement. Outbox retries delivery.

13. Inbox-callback topology

```

FakeBank / test provider
|
|   X-Internal-Service-Key
|
v
Gateway InternalService route
|
v
/api/v1/internal/bank/callbacks
|
v
Inbox dedupe (Source + MessageId)
|
v
Bank movement finalization

```

Repeated callbacks cannot create duplicate money movement because Inbox and terminal financial state are replay-safe.

14. Reconciliation topology

```

Internal Reconciliation API
|
+--> Wallet <-> Ledger-derived balance
|
+--> Bank transaction <-> Settlement ledger
|
+--> FinWallet <-> FakeBank statement
|
v
ReconciliationRun + ReconciliationIssue

```

Reconciliation reports differences but never silently mutates balances to make them match.

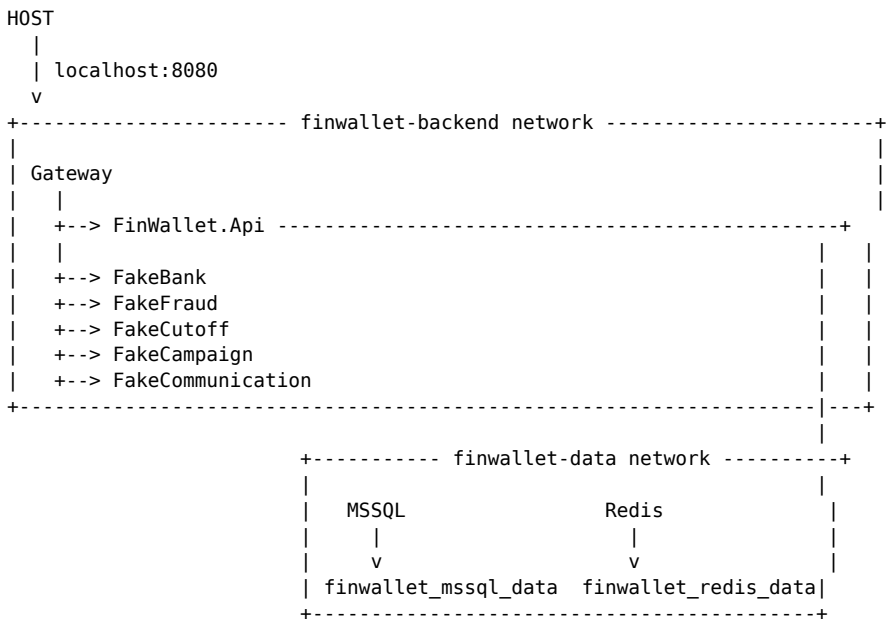
15. Swagger topology

Swagger is provided through FinWallet.Shared.Web for all HTTP projects:

```
FinWallet.Gateway
FinWallet.Api
FakeBank.Api
FakeFraud.Api
FakeCutoff.Api
FakeCampaign.Api
FakeCommunication.Api
```

It can be enabled in development and is disabled by default in production. Swagger visibility never bypasses endpoint authorization.

16. Docker runtime topology



Under normal Compose usage only Gateway 8080 is published to the host. Backend APIs, MSSQL and Redis remain inside Docker networks. The debug overlay exposes backend ports only through 127.0.0.1 when needed.

17. Docker persistent-volume topology

```
mssql container
+--> finwallet_mssql_data : database data/log
+--> finwallet_mssql_backup : backup files

redis container
+--> finwallet_redis_data : AOF/RDB persistence

application containers
+--> NO named persistent application volume
      stdout/stderr -> Docker log driver
```

docker compose down preserves volumes. docker compose down -v deletes local MSSQL and Redis persistence.

18. Scaling boundaries

Gateway, FinWallet.Api and stateless provider HTTP services can be replicated. Financial correctness must not depend on the number of replicas; MSSQL constraints, transactions and idempotency rules provide multi-instance correctness.

Redis is not used as a distributed financial lock. If an application instance disappears, committed financial state remains in MSSQL.

19. Financial-consistency summary

```
External HTTP
|
| SQL transaction is NOT open
v
Provider result
|
v
Short MSSQL transaction
|
+--> Wallet / BlockedBalance
+--> FinancialTransaction
+--> Ledger
+--> Idempotency
+--> Outbox
v
COMMIT
```

This boundary minimizes the need for distributed transactions and prevents partial financial commits.

20. References for tracing one request

End-to-end troubleshooting correlates the following references:

```
X-Correlation-Id
CustomerId
SessionId
TransactionId
BankReference / ExternalTransactionId
FraudReference
CampaignReference
CutoffReference
Outbox / Inbox message id
```

Tokens, OTPs, passwords, connection strings and other secrets are never logged.